Python: numpy

Intro. to computing

`https://mandarm.github.io/computing.html`

Background / motivation

- Many datasets can be represented as arrays (possibly multi-dimensional) of numbers.
- NumPy arrays (`ndarray`s) provide efficient storage and data operations (esp. compared to Python lists).
- Minor tradeoff: `ndarray`s are homogeneous, i.e., can only contain data of the same type.

Agenda

- Creating simple `ndarray`s
- Types supported by NumPy
- Some attributes of `ndarray`s
- More ways of creating `ndarray`s
- Operations on `ndarray`s

Creating NumPy arrays from Python sequences

```
import numpy as np
A = np.array([1, 4, 2, 5, 3])
B = np.array([1, 4, 2, 5, 3], dtype=float)
C = np.array([1.1, 4.2, 2.9, 5.4, 3.7], dtype=np.int32) # truncates!
D = np.array([range(i, i + 3) for i in [2, 4, 6]])

# All entries are ``promoted'' to the ``highest'' type required
E = np.array([1, 4, 2, 5, 3.14])      # integers promoted to floating
    points
C[1] = 10.1                           # C.dtype is integer => truncated
```

NumPy types I

- Types: Booleans, integers, floating points, complex numbers
- Similar to Python's types, but some **important differences**

  Python integers can be arbitrarily large but NumPy integers are *fixed width*

- **Signed integers**
  - `numpy.byte` / `numpy.int8`
  - `numpy.short` / `numpy.int16`
  - `numpy.intc` / `numpy.int32`
  - `numpy.int_` / `numpy.long` / `numpy.int64`
- **Unsigned integers:** for non-negative integers only
  - `numpy.ubyte` / `numpy.uint8`
  - `numpy.ushort` / `numpy.uint16`
  - `numpy.uintc` / `numpy.uint32`
  - `numpy.uint` / `numpy.ulong` / `numpy.uint64`

NumPy types II

- **Floats**
  - `numpy.single` / `numpy.float32`
  - `numpy.double` / `numpy.float64`
- NumPy maps standard Python type names to its own equivalents
  - `bool` → `numpy.bool` / `numpy.bool_`
  - `int` → `numpy.int_`
  - `float` → `numpy.double` / `numpy.float64`
  - `complex` → `numpy.complex128`

Terminology

- **axis** / **axes**: non-negative serial numbers assigned to dimensions
  e.g., `axis=0` for rows, `axis=1` for columns, `axis=2` for 3rd dimension
- **shape**: tuple of N non-negative integers specifying size of each dimension

Functions for creating `ndarrays` I

From Python sequences: see above

Converting existing arrays

- `np.copy(a)`: create a copy
- `np.astype(x, dtype)`: creates a copy with the specified type
- `np.fromiter(iterable, dtype, count)`: create a new 1-dimensional array from `iterable`

optional, but recommended so that required memory can be pre-allocated

Functions for creating `ndarrays` II

Creating special arrays: can optionally specify `dtype` in all of the following

- `np.empty(shape)`: create new *uninitialised* array of given `shape` and type (optional)
- `np.ones(shape)`: create new array of given shape, filled with ones
- `np.zeros(shape)`: create new array of given shape, filled with zeros
- `np.full(shape, fill_value)`: create new array of given shape, filled with `fill_value`
- `np.identity(n)`: create `n` $\times$ `n` identity matrix

  `np.eye(N, M, k)`: generalised version of above

Functions for creating `ndarrays` III

Numerical ranges (1-D arrays): can optionally specify `dtype` in all of the following

- `np.arange(start, stop, step)`: create array with evenly spaced values within given interval (like `range()`)
  - safest to use integers as `start`, `stop`, and `step` values
  - `stop` value may sometimes be included due to roundoff error
  - **Warning:** actual step value used to populate the array is `dtype(start + step) - dtype(start)`, rather than `step`, leading to counter-intuitive results.

```
np.arange(0, 5, 0.5, dtype=np.int_)
# -> array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0])
np.arange(-3, 3, 0.5, dtype=np.int_)
# -> array([-3, -2, -1,  0,  1,  2,  3,  4,  5,  6,  7,  8])
```

Functions for creating `ndarrays` IV

- `np.linspace(start, stop, num=50)`: create array with specified number of evenly spaced values in specified interval
- `np.logspace(start, stop, num=50, base=10.0)`: as above, but numbers are evenly spaced on a logarithmic scale in the interval `[base ** start, base ** stop]`
- `np.geomspace(start, stop, num)`: endpoints are `start` and `stop`, instead of `base ** start`, `base ** stop`

Reshaping

`A.reshape(new-shape-tuple)`

- Parentheses around tuple may be omitted.
- Sizes of initial array and reshaped array must match.

Functions for creating `ndarrays` V

Random matrices: returns single sample if `size` (shape) is omitted

- `np.random.random(size=None)`: create matrix of random floats from continuous uniform distribution over [0.0, 1.0)
- `np.random.uniform(low=0.0, high=1.0, size=None)`
- `np.random.normal(loc=0.0, scale=1.0, size=None)`
  `loc` $\equiv$ mean; `scale` $\equiv$ standard deviation
- `np.random.randint(low, high=None, size)`: create matrix of random integers from discrete uniform distribution over [`low`, `high`), or [0, `low`) (if high is omitted)

Functions for creating `ndarrays` VI

Recommended modern usage

```
rng = np.random.default_rng()
rng.random()
rng.uniform()
rng.normal()
rng.integers()  # NOTE new name!
```

Some attributes of `ndarrays`

`T`	view of the transposed array
`dtype`	data-type of the array's elements
`itemsize`	size of one array element in bytes
`nbytes`	total bytes consumed by array elements
`ndim`	number of dimensions
`shape`	tuple of sizes of dimensions
`size`	number of elements in the array

Views vs. copies

- **Indexing and slicing:** as for Python lists, except indexes / slices for multidimensional arrays can be separated by commas

  Examples:

```
A[i, :] # i-th row
A[i]    # i-th row
A[:, j] # j-th column
```

- **BUT** NumPy array slices are *always views*, **NOT** copies

  Examples:

```
a = np.array([1, 2, 3, 4, 5])
s = a[:]
s[2] = 499  # a changes
s = [0]     # Re-assignment!
s = a[:].copy()
s[2] = -999
```

```
l = [1, 2, 3, 4, 5]
l[:2] = []    # VIEW
s = l[:]      # COPY
```

Reshaping also (generally) returns no-copy view of initial array

Reference

Python Data Science Handbook, 2nd ed. (2023).

Jake VanderPlas, O'Reilly Media.

Part II, Chapters 4–12.